

BotSaaS

Documento de Arquitetura & Design

Plataforma SaaS de bot de IA para WhatsApp

C# .NET 10 · ASP.NET Core · MySQL · ADO.NET puro · Angular 21

Atualizado em 02/08/2026 · Full-stack: cliente conversa por chat (Telegram) → Agendamento gravado (function calling) → painel do dono em Angular (visão + gestão), testado e2e

Índice

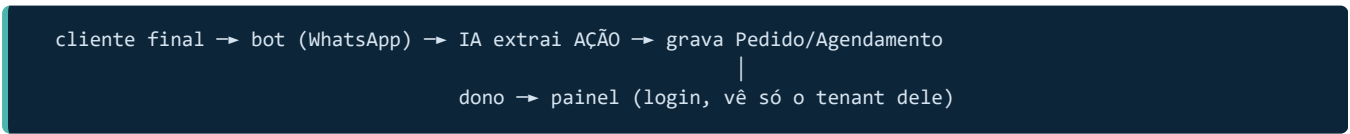
- 1. Visão do produto
- 2. Stack tecnológica
- 3. Arquitetura em camadas
- 4. Estrutura de pastas
- 5. Módulos atuais
- 6. Modelo de dados
- 7. Decisões de arquitetura
- 8. Fluxo: Registro (transacional)
- 9. Fluxo: Login (JWT)
- 10. Fluxo: Conversa (IA + memória)
- 11. Segurança
- 12. Injeção de dependência (DI)
- 13. Roadmap & estado atual
- 14. Pontos de atenção & próximos passos
- 15. Glossário

1. Visão do produto

SaaS de bot de IA para WhatsApp voltado a **pequenos negócios**. O cliente paga mensalidade e recebe um bot já configurado para o negócio dele, sem precisar saber de tecnologia.

Importante: o produto é uma **plataforma**, não "um bot". O bot é uma das funcionalidades dentro da plataforma. Cada loja é um *tenant* (uma *Company*), com um número, um bot e uma configuração própria.

O bot é TRANSACIONAL, não conversacional. A conversa é só a porta de entrada; o valor está em **virar ação estruturada**: o cliente final faz um **pedido** ou **marca um horário** pelo bot, e isso vira um **registro** que aparece num **painel para o dono do negócio**. O bot é o garçom; o produto é a comanda + a visão do dono.



Ator	Como usa	O que faz
Dono da loja	Painel web (Angular)	Cadastra-se, vê e gerencia os agendamentos (confirmar/concluir/cancelar), lê a conversa que gerou cada um. Futuro: configurar o bot, pagar a mensalidade
Cliente final	WhatsApp / Telegram	Manda mensagem para o bot da loja; a IA atende e registra a ação (pedido/horário)
Você (SaaS)	Infra	Hospeda o banco e a aplicação; responsável pelos dados (LGPD)

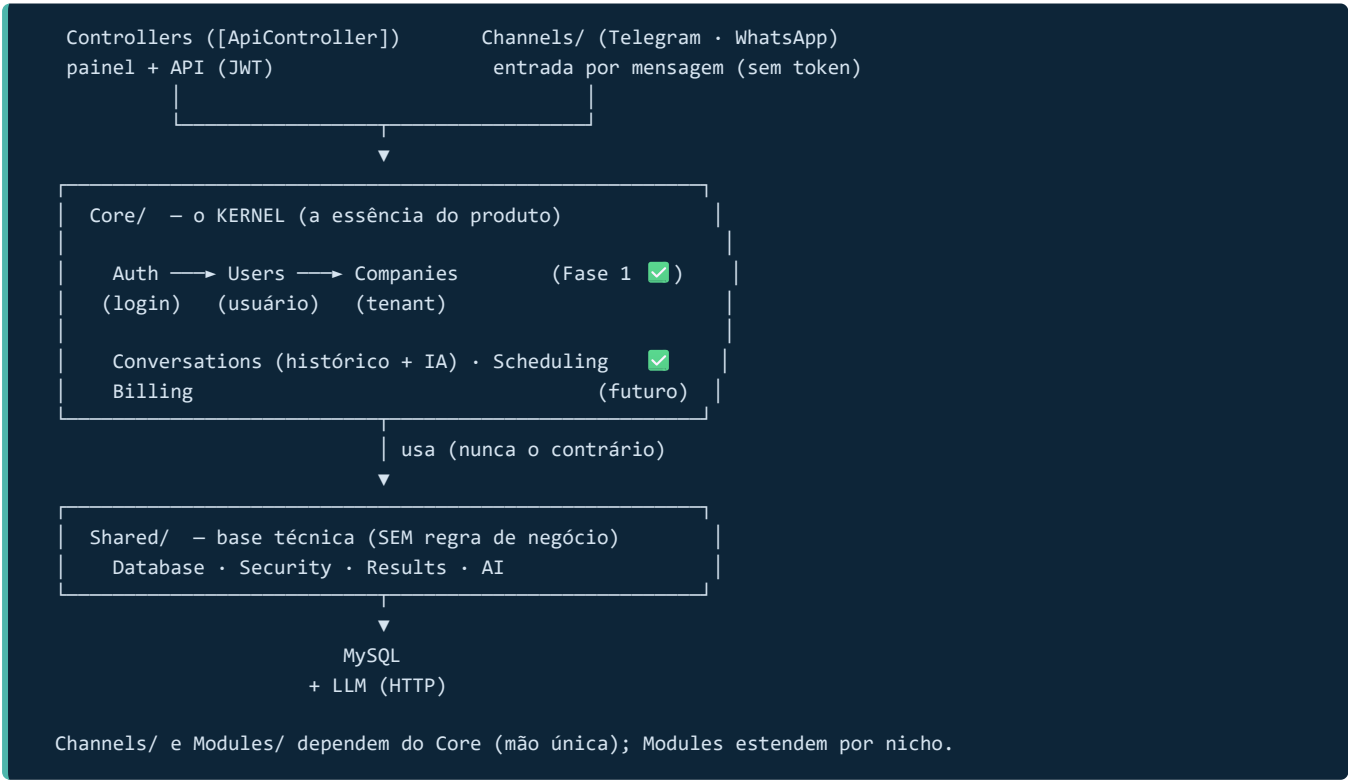
Contexto: projeto construído como peça de **portfólio** (demonstrar arquitetura + integração de IA end-to-end), com comercialização futura possível. Mira no fluxo end-to-end fechado como entregável principal.

2. Stack tecnológica

Camada	Escolha	Por quê
Backend	C# .NET 10, ASP.NET Core Web API (controllers)	Tipado, performático, ecossistema maduro
Banco	MySQL	Relacional, conhecido, barato de hospedar
Acesso a dados	ADO.NET puro + driver MySqlConnection	Sem ORM/Dapper — SQL escrito à mão, controle total, aprender o que está embaixo
Padrão de retorno	Result Pattern (mínimo)	Erro como valor, não como exceção espalhada
Auth	JWT próprio + BCrypt	Poucos usuários (donos), não precisa Auth0/Identity
IA	Google Gemini ou Groq/Llama (atrás de <code>IAiClient</code>)	O cérebro do bot — provedor trocável por DI (1 linha). Abstração já provada na prática
Frontend	Angular 21 (standalone, signals, zoneless)	O painel do dono. DI/decorators/services ecoam o backend C#; consome a API REST
Canais	Telegram (long polling, funcionando) · WhatsApp (webhook, futuro)	Adaptadores de entrada que roteiam a mensagem pro mesmo fluxo de conversa. Telegram é o canal de teste; WhatsApp é o alvo de produção
Config	.env (DotNetEnv)	Segredos fora do código

3. Arquitetura em camadas

Monolito **modular** (não microserviços). Três camadas com dependência de **mão única**:



Regra de ouro: Core depende de Shared, **nunca** o contrário. "Usado por todos" não significa "núcleo" — Shared é o *chão*, Core é o *centro*.

Onde mora a IA: a costura com o provedor de LLM (`IAiClient` + `GeminiAiClient` / `GroqAiClient`) fica em **Shared/AI** — é encanamento técnico neutro, sem regra de negócio, como `Database` e `Security`. Quem tem regra (montar prompt, salvar histórico, decidir a ação) é o **Core/Conversations**.

Como saber onde pôr cada coisa

- Fala de empresa / conversa / plano → **Core** (kernel).
- Serve qualquer app C# do mundo (Result, hash, conexão, cliente HTTP de LLM) → **Shared**.
- É específico de um nicho (barbearia, restaurante) → **Modules**.

4. Estrutura de pastas

```
src/
├── Channels/           ← adaptadores de entrada (chamam o Core)
│   └── Telegram/      (TelegramPollingService : BackgroundService, long polling · DTOs)
├── Core/              ← KERNEL
│   ├── Auth/          (autenticação: registro/login – orquestra o cadastro)
│   │   ├── AuthService.cs      IAuthService.cs
│   │   ├── AuthController.cs   AuthDTOs.cs (RegisterRequest, LoginRequest, UserResponse)
│   ├── Users/           (entidade User + gestão: perfil, listar)
│   │   ├── UserEntities.cs (User)      UserController.cs (GET /api/me)
│   │   ├── UserService.cs       IUserService.cs
│   │   ├── UserRepository.cs    IUserRepository.cs
│   ├── Companies/        (os tenants)
│   │   ├── CompaniesEntities.cs (Company, enum Segment)
│   │   ├── CompaniesService.cs   ICompaniesService.cs
│   │   ├── CompaniesRepository.cs ICompaniesRepository.cs
│   ├── Conversations/    (o fluxo do bot: conversa, memória, roteia tool calls)
│   │   ├── ConversationEntities.cs (Conversation, Message)
│   │   ├── ConversationService.cs  IConversationService.cs  ConversationDTOs.cs
│   │   ├── ConversationRepository.cs IConversationRepository.cs
│   │   ├── MessageRepository.cs    IMessageRepository.cs
│   ├── Scheduling/       (agendamentos – a ação transacional)
│   │   ├── SchedulingEntities.cs (Appointment, AppointmentStatus)
│   │   ├── SchedulingService.cs   ISchedulingService.cs
│   │   ├── AppointmentRepository.cs IAppointmentRepository.cs
│   ├── WhatsApp/ Billing/                                     ← futuro
├── Modules/            ← nichos que estendem o Core
│   └── Barber/          ← Fase 3
├── Shared/              ← base técnica
│   ├── Database/        (IDatabase, MySqlDatabase, DbSession, DbCommandExtensions)
│   ├── Security/        (IPasswordHasher, BcryptPasswordHasher, IJwtTokenGenerator, JwtTokenGenerator)
│   ├── Results/         (Result<T>)
│   └── AI/              (IAiClient, Gemini/GroqAiClient, AiResponse/TextReply/ToolCallReply, ToolDefinition, ChatMessa
```

Obs.: namespace é lógico (`BotSaaS.Api.Core.Users`), ignorando o `src/`. A pasta é organização; o **namespace** é o que define o módulo de verdade.

5. Módulos atuais

Módulo	Responsabilidade	Conteúdo	Estado
Auth	Autenticação — registro e login. Orquestra o cadastro (cria Company + User numa transação).	AuthService, AuthController, DTOs	e2e
Users	Dono da entidade User e da gestão (criar, buscar por email; rota <code>/api/me</code>). Futuro: perfil, listar.	User, UserService, UserRepository, UserController	e2e
Companies	Os tenants. Dono de "o que é uma Company válida". Futuro: BotConfig, settings.	Company, CompaniesService, CompaniesRepository	e2e
Conversations	O fluxo do bot: acha/cria a conversa do cliente, salva mensagens, monta o histórico, chama a IA e salva a resposta. Substitui a memória em RAM por persistência no banco (por cliente). Entrada via <code>ConversationController</code> (<code>POST /api/conversations</code> , protegido).	Conversation, Message, ConversationService, ConversationController, 2 repositórios	e2e
Scheduling	Os agendamentos — a ação transacional . Dono de "o que é um <code>Appointment</code> válido" (parseia data+hora, valida), da visão do dono (<code>GET /api/appointments</code>) e da gestão de status (<code>PATCH .../{id}/status</code> , tenant-scoped via <code>WHERE company_id</code>).	Appointment, AppointmentStatus, SchedulingService, AppointmentRepository, AppointmentController	e2e
Channels/Telegram	Canal de entrada (teste). Um <code>BackgroundService</code> faz <i>long polling</i> no Telegram e roteia cada mensagem pelo mesmo <code>ConversationService</code> . O tenant é resolvido por config (<code>.env</code>); em produção, webhook + tabela <code>Channel</code> .	TelegramPollingService, TelegramDtos	e2e
Shared/AI	Costura neutra com o LLM. <code>IAiClient</code> esconde o provedor; traduz <code>systemPrompt</code> +histórico+ tools , e devolve texto ou tool call (<code>AIResponse</code>).	IAiClient, Gemini/GroqAiClient, AiResponse, ToolDefinition, ChatMessage	e2e

Por que Auth e Users são separados? Autenticação (login/registro/senha) é um concern; a entidade User e sua gestão (perfil, listar) é outro. `Auth` **depende de** `Users` , mão única.

6. Modelo de dados

Relações

Company 1—N User	✅ existe (tenant + donos)
Company 1—N Conversation	✅ existe (por cliente/telefone)
Conversation 1—N Message	✅ existe (Role: User / Assistant)
Conversation 1—N Appointment	✅ existe (ação extraída pela IA)
Company 1—N Appointment	✅ existe (tenant)
Company 1—N Channel	⌚ futuro (Telegram já funciona via config; a tabela vem no onboarding)

Simplificação atual: a `Conversation` nasceu enxuta, *sem* a entidade `Channel` ainda. O telefone do cliente (`customer_phone`) vai direto na conversa; o par (`company_id`, `customer_phone`) identifica o fio. Quando o

`Channel` existir, isso vira `ChannelId` + `ContactIdentifier` (agnóstico de canal). Sem `LastMessageAt` / `IsActive` ainda — expiração por inatividade vem depois.

Tabelas (criadas)

companies		users		conversations	
id	CHAR(36) PK	id	CHAR(36) PK	id	CHAR(36) PK
name	VARCHAR(150)	company_id	CHAR(36) FK	company_id	CHAR(36) FK → companies
segment	INT	name	VARCHAR(150)	customer_phone	VARCHAR
created_at	DATETIME	email	VARCHAR(255) U	created_at	DATETIME
		password_hash	VARCHAR(255)		
		created_at	DATETIME		
				messages	
				id	CHAR(36) PK
				conversation_id	CHAR(36) FK → conversations
				role	INT (reusa ChatRole)
				content	TEXT
				created_at	DATETIME(6) ◀ microssegundo
appointments (ação transacional que a IA extrai da conversa)					
id	CHAR(36) PK				
company_id	CHAR(36) FK → companies				
conversation_id	CHAR(36) FK → conversations				
service_name	VARCHAR(150)				
customer_name	VARCHAR(150)				
scheduled_at	DATETIME				(naive / hora local do negócio)
status	INT				(AppointmentStatus: Pending=0, Confirmed=1, Cancelled=2, Completed=3)
created_at	DATETIME				(UTC)

email UNIQUE global: um e-mail = uma conta. É essa constraint que gera o erro `1062` (duplicate) no cadastro repetido — e que o teste de atomicidade usou para provar o rollback.

messages.role é INT, não string: reusa o mesmo enum `ChatRole` (`User=0` / `Assistant=1`) do módulo de IA. Mesma regra de todo enum no banco — nunca reordenar os números depois que tiver dado.

messages.created_at é DATETIME(6) (microssegundo): numa conversa rápida (ex: Telegram), a mensagem do cliente e a resposta do bot caem no mesmo segundo; com resolução de 1s elas empatavam e o `ORDER BY created_at` desempata na sorte (PK Guid aleatório), bagunçando a ordem no painel. A precisão de sub-segundo — que o app já insere via `DateTime.UtcNow` — resolve.

appointments — o registro transacional: é o que a IA extrai da conversa (via function calling) e grava — o "produto". `scheduled_at` é **naive / hora local do negócio** (o "09:00" que o cliente falou); `created_at` é **UTC**. São **frames diferentes** — não comparar os dois direto (por isso ainda sem validação de "data futura"). Multi-fuso real (guardar o fuso da empresa) é futuro.

Mapeamento C# → MySQL

C#	MySQL	Detalhe
Guid	CHAR(36)	Texto legível; evita conversão de bytes do BINARY(16) no ADO.NET puro. Ler com <code>(Guid)reader["id"]</code> — o driver já entrega Guid, nunca <code>Guid.Parse</code> (estoura <code>InvalidCastException</code>)
enum	INT	Valor numérico; <code>(int)e</code> ↔ <code>(Enum)reader.GetInt32()</code> . Nunca reordenar os números.
DateTime	DATETIME	Preenchido pela aplicação (<code>DateTime.UtcNow</code>), não pelo banco
Coluna: <code>snake_case</code> sem prefixo (<code>created_at</code> , não <code>comp_created_at</code>). SELECT sempre com colunas explícitas.		

7. Decisões de arquitetura

7.1 Repositório burro (Escola A)

O repository tem **uma** responsabilidade: falar SQL. Ele **roda SQL ou estoura** a exceção do driver. Sem `try/catch`, sem `Result` lá dentro. Retorna dado cru, nunca `Result`.

- Insert/Update/Delete → `Task`
- Buscar um → `Task<User?>` (`null` = não achou, caso normal)
- Buscar vários → `Task<List<T>>` (lista vazia, nunca `null`)

7.2 Camadas de erro (contrato)

```
Repository → estoura a exceção do driver (sobe como DbException)
|
Service → captura DbException (base abstrata, NÃO a MySqlConnection concreta),
|         valida regra, devolve Result com código semântico
|         [não conhece HTTP nem idioma]
|
Controller → traduz o código em status HTTP (400/401/404/409) + mensagem
|         [único que conhece HTTP]
```

Capturar `DbException` (de `System.Data.Common`) e não `MySqlException` mantém o **Core desacoplado do driver** — mesmo princípio do `IDatabase`.

7.3 Atomicidade do registro = transação via DbSession

Registrar cria **Company + User**: ou nascem os dois, ou nenhum. Como uma transação vive numa **única conexão**, os dois INSERTs precisam compartilhá-la.

- `DbSession` (Scoped) — caixa que segura `Connection` + `Transaction` da requisição.
- `AuthService` é o **dono do ciclo**: cria a conexão, abre, `BeginTransaction`, enche o `DbSession`, e no fim `Commit` (ou `Rollback` no catch).
- Os repos injetam o **mesmo** `DbSession` (Scoped = mesma instância) e leem dele → os 2 INSERTs caem na mesma transação sem passar nada por parâmetro.

Descartado por ora: passar a transação por parâmetro (mais explícito, repete) e Unit of Work com comportamento. Quando vários services dividirem transação, o `DbSession` vira UoW.

7.4 Interface neutra (esconder a tecnologia)

Toda tecnologia específica fica atrás de uma **interface com nome neutro**; a marca aparece só na implementação:

Interface (neutra)	Implementação (marca)	Esconde
IDatabase	MySqlDatabase	MySqlConnection
IPasswordHasher	BcryptPasswordHasher	BCrypt
IJwtTokenGenerator	JwtTokenGenerator	System.IdentityModel.Tokens.Jwt
IAiClient	GeminiAiClient / GroqAiClient	Google Gemini / Groq (Llama)

Trocar a tecnologia (MySQL→Postgres, BCrypt→Argon2, Gemini→Groq) custa **uma linha na DI**.

7.5 IA neutra — um contrato, dois provedores

O contrato é único e burro: `Task<AiResponse> GenerateReplyAsync(string systemPrompt, IReadOnlyList<ChatMessage> history, IReadOnlyList<ToolDefinition> tools)`. O client recebe o prompt-base, o histórico e as ferramentas disponíveis, e devolve um `AiResponse` — que é **ou um texto** (`TextReply`) **ou um pedido de ferramenta** (`ToolCallReply`). Quem monta o prompt, define as tools e persiste o histórico é o Core. Cada implementação traduz isso pro formato da sua API — e as diferenças **provam** o valor da abstração:

	Gemini	Groq (formato OpenAI)
Histórico	<code>contents</code>	<code>messages</code>
Papel do assistente	<code>"model"</code>	<code>"assistant"</code>
System prompt	campo separado <code>system_instruction</code>	1ª mensagem <code>role:"system"</code>
Conteúdo	<code>parts[].text</code>	<code>content</code> (string)
Chave da API	na URL (<code>?key=</code>)	header <code>Authorization: Bearer</code>
Resposta	<code>candidates[0].content.parts[0].text</code>	<code>choices[0].message.content</code>

Provado em teste e2e: os dois provedores respondem 200 seguindo o system prompt. Trocar Gemini ↔ Groq é **uma linha** no `Program.cs` (`AddHttpClient<IAiClient, X>`) — nenhum caller muda. É a prova concreta de que a abstração não é enfeite.

LLM é stateless: o modelo não tem memória. "Memória" = reenviar o system prompt + o histórico inteiro a cada chamada. Por isso o histórico precisa viver em algum lugar (o banco) e ter um teto (senão o custo e a latência crescem sem limite).

Function calling — o coração transacional (provado e2e): o contrato evoluiu de `Task<string>` para `Task<AiResponse>` para o modelo poder, além de responder texto, **pedir uma ação estruturada** (`registrar_agendamento` com `servico/data/hora/nome`). As ferramentas vêm do **Core** (tipos neutros `ToolDefinition` / `ToolParameter` , por empresa/nicho no futuro); o client só **formata** pro provedor (Groq: `tools` / `tool_calls`). Tool = **ação/verbo**; o serviço é um **parâmetro**, não uma tool por item. **Provado e2e no Groq:** o modelo extraiu os args e o `Core/Scheduling` gravou o `Appointment` — conversa vira ação. A confirmação hoje é **templada**; o loop multi-turno (modelo frasear) e o tool calling do Gemini ficam pra depois.

7.6 System prompt dinâmico (por empresa)

O prompt final é **por tenant**: cada empresa gera o seu a partir do `BotConfig` (nome, segmento, horário, serviços, tom). Hoje o prompt vem de fora, cru (do `.env`); o desenho-alvo é **base (template) + dados da empresa (banco)** montados em runtime. O `IAiClient` é burro: só recebe o prompt pronto e manda. Cada nicho poderá contribuir com contexto extra via `INichoConfig.BuildSystemPromptExtension()`.

8. Fluxo: Registro (transacional)

```
POST /api/auth/register { ownerName, email, password, companyName, segment }
|
v
AuthController ..... valida HTTP, chama o service
|
v
AuthService.Register ..... DONO da transação
|
| 1) cria conexão (IDatabase) → enche DbSession → OpenAsync → BeginTransaction
|
| → CompaniesService.CreateCompany(name, segment)
|   ↳ CompaniesRepository.InsertCompany → INSERT companies
|
| → UserService.CreateUser(name, email, password, companyId)
|   ↳ hash da senha (IPasswordHasher.Hash → BCrypt)
|   ↳ UserRepository.InsertUser → INSERT users
|
| 2) Commit (deu tudo certo)
|   catch DbException → Rollback (desfaz tudo, sem Company órfã)
|
v
Result<User> → Controller mapeia para UserResponse (SEM o hash) → 200 OK
```

Provado em teste e2e: registro → 200; email repetido → 400 + rollback confirmado no banco (nenhuma Company órfã). A atomicidade funciona.

9. Fluxo: Login (JWT)

```
POST /api/auth/login { email, password }
|
v
AuthController
|
v
AuthService.Login ..... leitura, SEM transação
|
| abre conexão (enche DbSession.Connection, sem BeginTransaction)
|
| UserService.GetUserByEmail(email) → UserRepository → SELECT users
|   ↳ null? → Result.Failure("Credenciais inválidas")
|
| IPasswordHasher.Verify(senha, hash)
|   ↳ inválida? → Result.Failure("Credenciais inválidas") ← MESMA msg (não revela qual)
|
| IJwtTokenGenerator.GenerateToken(userId, companyId, email)
|
| CompaniesService.GetCompanyId(companyId) — nome da empresa (white-label)
|
v
Result<LoginResult> (token + companyName) → 200 { token, companyName } | falha → 401
```

Provado em teste e2e: senha certa → 200 + JWT; senha errada e email inexistente → 401. Rota protegida `GET /api/me` lê `sub / companyId / email` das claims; token adulterado → 401 (a assinatura HMAC barra a forja).

10. Fluxo: Conversa (IA + memória)

O coração transacional do produto. Substitui a antiga memória em RAM (uma lista global, deletada) por histórico **persistido no banco, por cliente**.

```
(dev)      POST /api/conversations { customerPhone, messageText } [Authorize]
          └─ companyId vem do JWT (claim), NÃO do body
(canal)    Telegram (long polling) → chatId vira o "customer_phone"; companyId por config (.env)
(produção) Webhook do WhatsApp    → sem token; companyId vem do CANAL (tabela Channel)
          │
          ▼
ConversationService.ProcessMessage
├─ find-or-create: GetConversationByCompanyAndPhone(company, phone)
│   └─ null? → cria a Conversation (cliente novo)
├─ InsertMessage (msg do cliente, Role=User)
├─ GetMessagesByConversation → carrega histórico (ORDER BY created_at)
├─ converte List<Message> → List<ChatMessage> · monta as tools (registrar_agendamento)
├─ IAIClient.GenerateReplyAsync(systemPrompt, history, tools) → LLM (Groq)
│   └─ TextReply → resposta = texto
│       └─ ToolCallReply → parseia args → SchedulingService.CreateAppointment
│           └─ grava Appointment · resposta = confirmação templada
├─ InsertMessage (resposta, Role=Assistant)
          ▼
Result<string> (a resposta) → devolve ao cliente
```

Por que "find-or-create": o `null` do repositório separa a 1ª mensagem (cria a conversa) das demais (só insere a msg). O par `(company_id, customer_phone)` respeita o multi-tenant — a conversa é sempre buscada dentro do tenant.

Provado em teste e2e (Groq): register → login (token) → `POST /api/conversations` duas vezes na mesma conversa. O bot lembrou do pedido dito no 1º turno (memória via histórico no banco) e seguiu o system prompt (disse que ia "verificar o preço" em vez de inventar). Cadeia completa: token → tenant do JWT → service → LLM → grava → resposta.

Transacional provado e2e (Groq): "quero agendar um corte de cabelo para 2026-08-05 às 09:00, meu nome é Breno" → o modelo chamou `registrar_agendamento` com os 4 args certos → o `SchedulingService` gravou o `Appointment` (linha confirmada na tabela `appointments`) → confirmação templada de volta. **A conversa virou ação estruturada** — o bot deixou de tagarelar e passou a registrar.

Entrada atual vs. produção: hoje a entrada é o `ConversationController` protegido — o `companyId` sai do **JWT** (correto para uma entrada com token), e o body só traz `customerPhone` + `messageText`. É marcada como **dev/token** porque a entrada de produção é o **webhook do WhatsApp**, onde **não há token**: ali o `companyId` virá do **canal** que recebeu a mensagem (qual número/instância).

Feito

Entidades User/Company

Banco + .env + DI

Result<T>

DbSession + transação

Registro e2e

Hash BCrypt

JWT + /api/me

Login e2e

IAiClient (Gemini + Groq) e2e

Conversations (fluxo completo) e2e

Histórico no banco por cliente

Conexão fora do LLM

Validação de entrada (DataAnnotations)

Function calling + Scheduling e2e

Agendamento transacional (conversa → banco)

Visão do dono (GET /appointments) e2e

Painel do dono em Angular (login + tabela)

Gestão de status (PATCH) + botões e2e

Link agendamento → conversa (chat) e2e

Data no prompt + tool tunada (data/hora natural)

git + README + .gitignore

Canal Telegram (long polling) e2e

Painel repaginado (shell/sidebar)

White-label (marca do tenant no login)

Ordenação de mensagens (DATETIME(6))

Em construção / a endurecer

Loop multi-turno (confirmação frasada pelo modelo)

Teto de histórico

Expiração por inatividade

Gemini tool calling

Webhook (troca o polling do Telegram)

Slot-filling (garantir dados antes do tool)

14. Pontos de atenção & próximos passos

Pontos de atenção conhecidos (Conversations)

- **Conexão presa durante o LLM** — ☒ **resolvido**. O `ProcessMessage` abre a conexão em **dois trechos curtos** (escrita antes / escrita depois), com o LLM **fora** de qualquer conexão aberta — o pool não fica preso sob carga.
- **"Erro" persistido** — ☒ **resolvido**. Agora `GenerateReplyAsync` **estoura** `AiClientException` em status ruim/null (não devolve string de erro). O `ConversationService` captura (`catch(AiClientException)`) e devolve `Result.Failure` — nenhuma resposta falsa é gravada. Como a escrita é autocommit, a mensagem do cliente sobrevive à falha da IA.
- **Teto de histórico**. `GetMessagesByConversation` traz *todas* as mensagens; sem limite, o prompt cresce sem parar (custo/latência/contexto). Precisa de teto (N mensagens ou janela de tokens).
- **Expiração por inatividade**. Sem `LastMessageAt` / `IsActive`, a conversa nunca "fecha". Definido: expira por inatividade (4–8h), não por tempo fixo.

Próximos passos

1. **Extração de ação (function calling)** — ☒ **feito e testado e2e**. A IA chama `registrar_agendamento` e o `Core/Scheduling` grava o `Appointment`. Restante: loop multi-turno (modelo frasear a confirmação) e tool calling do Gemini.
2. **Endurecer o fluxo de Conversa** — teto de histórico e expiração por inatividade (conexão-fora-do-LLM e tratamento de falha da IA já feitos).
3. **Pacote de erro** — `Result.Error` virar código semântico; "email já existe" → HTTP **409** (`DbException.SqlState=="23000"` ou `MySqlException.Number==1062`).
4. **Canal de WhatsApp** — webhook + entidade `Channel`; `companyId` a partir do canal (não do body). Idempotência por message-id (WhatsApp reentrega).
5. **Painel do dono** — ler os `Appointment` do banco (a "visão do dono") — primeira parte visível do produto.

TODOs no código

- `AuthService` ainda devolve `Result.Failure("Erro ao registrar")` (string) → código semântico.
- Leitura das envs JWT repetida (gerador + validador) → centralizar em options.
- **Posse do tool** `registrar_agendamento` está na `Conversations` (declaração + args + handler); mover pra `Scheduling` e deixar a `Conversations` como roteador de tool calls — TODO no código, fazer no 2º nicho.

- `MessageRepository.GetMessagesByConversation` passa o `@conversationId` como `Guid` cru, enquanto os outros repos passam `.ToString()` — inconsistência (funciona, mas depende do formato de Guid do driver).
- **Canal Telegram por polling é temporário** — trocar por **webhook** em produção (é o que o WhatsApp Cloud API exige; são mutuamente exclusivos). O reaproveitável entre os dois é o `ProcessMessage`.
- **Resolução de tenant no canal via `.env`** (`TELEGRAM_TEST_COMPANY_ID`) é provisória. Multi-tenant real = tabela `Channel` (`type`, `external_id`, `token` → `company_id`), populada no onboarding.

15. Glossário

Termo	O que é
Multi-tenant	Vários clientes (lojas) no mesmo banco, isolados por <code>CompanyId</code> .
Transacional (o bot)	A conversa vira uma ação estruturada (pedido/agendamento) gravada, não só troca de texto.
Transação (ACID)	Conjunto de operações "tudo ou nada": <code>BEGIN</code> → <code>COMMIT</code> (grava) ou <code>ROLLBACK</code> (desfaz).
DI (Injeção de Dependência)	Receber colaboradores prontos em vez de criá-los; o container monta tudo.
Repository	Camada que só fala SQL com o banco. "Burra" — sem regra de negócio.
Service	Camada de domínio: regras, orquestração. Não conhece HTTP.
DTO	Objeto de transporte na borda (request/response ou entre camadas). Ex: <code>RegisterRequest</code> , <code>ChatMessage</code> .
<code>Result<T></code>	Embrulho de retorno: sucesso (com valor) ou falha (com erro). Genérico — <code>T</code> é o tipo do valor.
Hash (BCrypt)	Transformação irreversível da senha, com salt. Não dá pra "descriptografar".
JWT	Token assinado que prova quem é o usuário sem consultar o banco a cada request.
Claim	Uma afirmação dentro do token (id, companyId, email).
LLM stateless	O modelo não guarda memória. "Memória" = reenviar system prompt + histórico a cada chamada.
System prompt	Instrução-base que define o comportamento do bot; no BotSaaS, dinâmico por empresa.
Function calling	Recurso onde o modelo decide chamar uma "função" (ação) da app — o mecanismo pra extrair pedido/agendamento.